Applications of Machine Learning

Applicazioni Informatiche del Machine Learning

LECTURE 1 | GÉRON CH. 1–2

What machine learning is, and how we will work

BSc Mathematics of Artificial Intelligence · Third year

48 academic hours · 24 lectures

Textbook: Géron, *Hands-On Machine Learning with Scikit-Learn and PyTorch*

The course in one sentence

How to build a machine learning system,
and how to know whether it works.

> **THE SECOND HALF IS THE HARD HALF**
>
> Fitting a model is a few lines. Establishing that the number it reports means what it appears to mean is the whole of the rest of the course.

What this course is

- An **applied** course: every method arrives with a dataset, a measurement and a worked example
- A course about **knowing whether your system works** — evaluation is the spine
- A course with the **mathematics kept in**: each lecture derives the one object its method rests on
- A course that ends with two topics the textbook does not cover — **search** and **recommendation**

What this course is ✗ not

- Not a survey of machine learning — we cover one book thoroughly, and then two topics properly
- Not a software-engineering course — you will not build production infrastructure
- Not a typing course — you will use AI assistance throughout, and we will spend today on how to do that well
- Not a course where a working notebook earns marks. **✗ Evidence earns marks.**

The textbook

Aurélien Géron

Hands-On Machine Learning with Scikit-Learn and PyTorch

O'Reilly, 2025

> *The book is the syllabus.*
> *The notebooks are ours.*

Chapters 1–16, covering Lectures 1–18 and 23–24.

We do not use anyone else's notebooks — not even the author's. Every notebook in this course is written for it, and every one of them is complete: nothing in them is wrong on purpose.

EXAMINABLE SCOPE

Chapters 1–16, **plus** the lecture notes for Lectures 19–22 on information retrieval and recommender systems, which sit outside the book. Where the field goes beyond both, I will say so and move on.

Twenty-four lectures, six parts

Part	Lectures	What
I · Tabular data and classical models	1–8	regression, classification, trees, ensembles, PCA, clustering
II · Neural networks	9–11	from the perceptron to PyTorch to deep training
III · Computer vision	12–14	convolution, transfer learning, detection
IV · Sequences and language	15–18	forecasting, recurrent networks, text, transformers
V · Search and recommendation	19–22	retrieval and recommender systems — beyond the book
VI · Multimodal models	23–24	vision transformers, dual encoders, generation

One topic per lecture, in the order of the book. Each lecture stands on its own: if you miss one, you read its deck and its notebook and you are caught up.

The shape of every lecture

min	What
0–10	where we are, and what today's method is for
10–30	**the mathematics** — the one object the method rests on, derived
30–70	**the method** — how it works, its hyperparameters, what breaks it, and a worked example with real numbers
70–85	variants, and what is actually used in practice
85–90	the notebook — what is in it and what to do with it

> **THE NOTEBOOKS ARE FOR YOU, NOT FOR THE LECTURE**
>
> Every lecture ships a complete, commented, runnable Colab notebook. We will not build it live — we will look at what is in it, and you run it afterwards.

The mathematics is not an appendix

Each lecture opens by deriving the one object its method rests on. A sample of what is coming:

- Least squares and the normal equation · L2
- Why precision is not monotone in the threshold · L3
- The bias–variance decomposition · L5
- Variance of an average of correlated predictors · L7
- PCA via the SVD; Johnson–Lindenstrauss · L8
- Backpropagation as reverse-mode autodiff · L10
- Variance propagation and initialisation · L11
- Softmax, cross-entropy and logits · L17
- Scaled dot-product attention · L18
- The contrastive objective and its temperature · L23

✗ Eighteen derivations in all, and they are 40% of the written paper. They are also the part you will still have in five years, when the libraries have all changed.

Working method: AI-assisted development

You will write machine learning code with an assistant — if not in this course, then in the thesis after it and in the job after that. So we are going to be explicit about how, rather than pretend it is not happening.

> **WHY THIS IS WORTH TWENTY MINUTES OF LECTURE 1**
>
> Applied machine learning is a small amount of library glue and an enormous amount of data work and evaluation. The scarce skill is **judgement about whether a result can be trusted**, not typing — and an assistant supplies the typing and none of the judgement.

The notebooks in this course are written for you and they are correct. What you have to learn is what to do when neither of those is true.

Four rules for AI-assisted work

1. **Never keep code you cannot explain line by line.** Part B of the paper puts a cell in front of you and asks what would have to be true for its number to be trusted
2. **Every number needs a baseline.** A metric with nothing to compare it to is decoration
3. **Every model needs an ablation.** Remove a component; show the number moves
4. **Report the failure cases.** A system with no known failure mode has not been tested

Rules 2 to 4 are what a result has to come with before anyone should believe it — yours or an assistant's. They are also, roughly, Part C of the written paper.

Why those rules: ML fails *silently*

A bug in a training loop does not crash. It gives you a plausible number.

- Scaler fitted before the split → leakage → great score, broken system
- Missing `model.eval()` → dropout still active during evaluation
- Missing `optimizer.zero_grad()` → gradients accumulate, no error
- Metric averaged per batch instead of over the set → wrong by construction
- Accuracy on an imbalanced set → 90% from a model that predicts one class

✗ Every one of these runs. Every one of these produces a number.

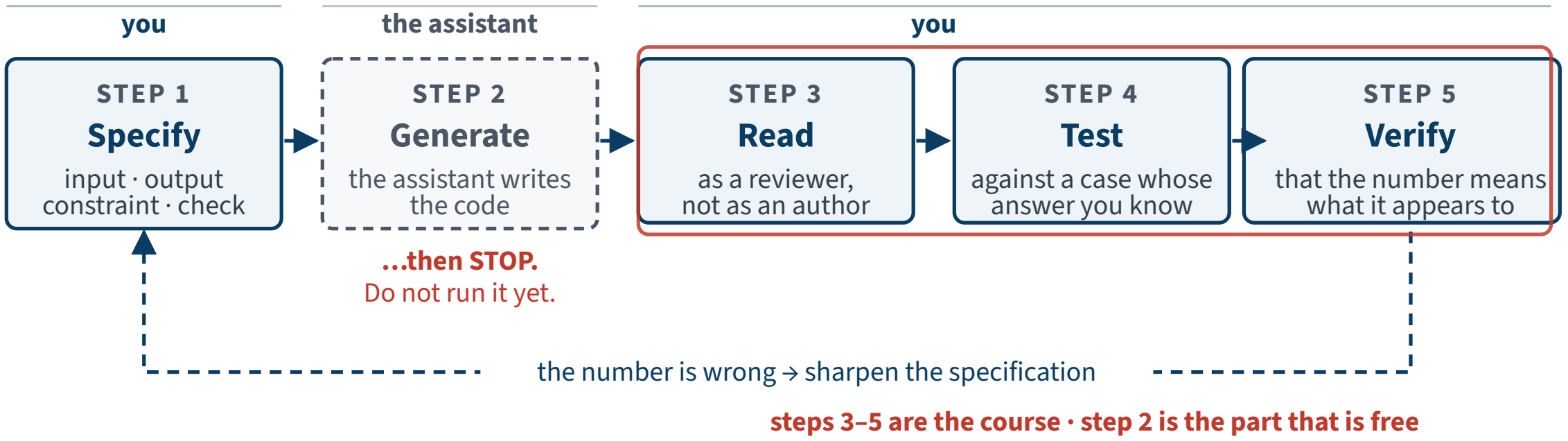
The loop

Not “ask for code and run it”. Five steps, and only one of them belongs to the assistant:

#	Step	Who does it
1	**Specify** — state the task, the data, the constraint	you
2	**Generate** — and then *stop*	the assistant
3	**Read** — as a reviewer, not as an author	you
4	**Test** — against a case whose answer you know	you
5	**Verify** — that the number means what you think	you

✗ Steps 3–5 are the course. Step 2 is the part that is free.

The loop



Four of the five steps are yours. The one an assistant does for you is the one that was never the hard part.

Step 1 — Specify

A usable specification names four things:

1. **The input** — shape, types, where it comes from
2. **The output** — exactly what object, with what guarantees
3. **The constraint** — what must *not* happen
4. **The check** — how you will know it worked

> **THE ONE PEOPLE ALWAYS OMIT**
>
> Number 3. Almost every silent failure in machine learning is a missing constraint, not a missing feature.

What a usable specification looks like

The task: prepare the housing features for a model. Written out, all four parts:

input · the 16,512-row training frame, 8 numeric columns and 1 categorical, with 168 missing values in `total_bedrooms`

output · a fitted `ColumnTransformer` and the transformed matrix

constraint · the imputer and the scaler are fitted on the *training rows only*; the test rows are transformed with those statistics and never contribute to them

check · the output has 16,512 rows and 13 columns — 8 numeric plus 5 one-hot levels — and no NaN

WHERE YOU WILL MEET THIS LOOP IN THIS COURSE

Every code cell in every notebook is preceded by a box in exactly this form. The box is **step 1**, the cell below it is what **step 2** returned, and the `check` line is **step 4** written down in advance. Read the box, answer the check in your head, *then* run the cell — and you have done the whole loop with only the free step handed to you.

Step 2 — Generate, then *stop*

The dangerous moment is not the generation. It is the reflex to run it.

> **THE RULE**
>
> Generated code is **read before it is executed**. Output you have already seen is very hard to disbelieve.

This is not caution for its own sake. In a domain where wrong code still returns a plausible number, running first destroys the only cheap opportunity to notice.

Step 3 — Read it like a reviewer

Five questions, asked in this order, every time:

1. **What touched the test set?** Trace every line backwards to its data source
2. **What was fitted, and on what?** `fit` and `transform` are different verbs
3. **What is the shape here?** Silent broadcasting is silent
4. **What was dropped?** Rows, columns, NaNs — count them
5. **What is the default I did not ask for?** Every unnamed argument is a decision someone else made

Step 4 — Test against a case you already know

Not “does it run”. **Does it give the right answer on an input whose answer you can state independently?**

- **A shape.** 16,512 rows in, 16,512 rows out — and if one-hot encoding added 5 columns, 13 out, not 9
- **A count.** 168 missing values before, 0 after
- **A degenerate case.** A constant column has zero variance; what does the scaler do with it?
- **Arithmetic you can do on paper.** A layer with 32 inputs and 32 outputs has $32 \times 32 + 32 = 1{,}056$ parameters. Print the number and compare

> **THE RULE**
>
> If you cannot state the expected answer before running the cell, you are not testing — you are watching.

Step 5 — Verify that the number means what it appears to

The code is correct and the number is still misleading. This is the step nobody does.

- **Against what?** A metric with nothing beside it is decoration. What does the trivial model score?
- **On which rows?** Training, validation or test — and were they chosen before or after anything was fitted?
- **How much does it move?** A difference smaller than the spread across splits has not been measured
- **In what units?** An error of 70,000 is meaningless until you know the prices run from 15,000 to 500,000

✗ Every one of the four is a question about evidence, not about code. That is the course.

Assessment

Written examination — carries the mark

- Two hours, closed book, **no formula sheet**
- Part A — the derivations (40% of the paper)
- Part B — choosing a method for a stated situation (35%)
- Part C — reading results: curves, matrices, metrics (25%)

Oral examination — optional

- Seven to ten minutes, no notes, no computer
- **Three questions drawn in front of you** from the 120 exercises that end these decks
- You decide after seeing your written mark

> **HOW THE MARK IS MADE**
>
> Written marked out of 30, pass at 18, and **on its own capped at 27**. The oral moves that mark by **at most ±3** and the result is final: 27 can become 30, and it can become 24. It cannot turn a passing written into a fail. Binding once registered.

✗ Note what is absent: no marks for a notebook that runs. Evidence earns marks.

“A written exam, on an applied course?”

Because what you are learning is not typing. It is **judgement about whether a result can be trusted** — and you exercise that by reading: code you did not write, a metric, a curve. At a keyboard you can reach the right answer by running things until they look right. On paper you have to actually know.

1. **No question tests API recall.** In a course where you deliberately do not type the code, asking for the arguments of `train_test_split` would be incoherent. Syntax errors in handwritten code cost nothing; logic errors cost everything
2. **No question is answerable by reciting a definition.** Every one applies it to a stated situation
3. **The paper states what it needs.** Any definition a question depends on is printed in the question — which is why there is no formula sheet and why you will not miss one

> **WHAT PART B ACTUALLY LOOKS LIKE**
>
> A situation, described in four lines: this data, this constraint, this deadline. Which method, and what would make you change your mind? The specimen on the next slide is one.

A specimen Part B question

```
best = None
for a in [0.01, 0.1, 1, 10, 100]:
    m = Ridge(alpha=a).fit(X_train, y_train)
    s = root_mean_squared_error(y_test, m.predict(X_test))
    if best is None or s < best[1]:
        best = (a, s)

print(f"best alpha={best[0]}, test RMSE={best[1]:,.0f}")
```

1. This program never creates a validation set. **Which object is doing that job?**
2. As an estimate of the model's error on new data, the printed RMSE is **(i)** too optimistic, **(ii)** too pessimistic, or **(iii)** neither. Choose one, and give the reason in a sentence.
3. Rewrite the program so the printed number is honest. Two statements is enough.
4. In expectation over repeated train/test splits, is the honest number **higher** or **lower**? Is that guaranteed on any *single* split?

Specimen answers — 1 and 2

1. Which object is the validation set?

✓ The test set.

- Line 4 scores on it; line 5 uses that score to *choose* α. Choosing is a validation set's job
- Once it has been used to choose, no held-out data remains

2. Too optimistic, too pessimistic, or neither?

✓ (i) too optimistic.

- The printed value is the **minimum** of five numbers measured on the same data used to report it
- α was selected *because* it scored lowest there, so the score cannot also be a fair test of it

Questions 3 and 4 — the repair, and whether the honest number is higher — are answered at the end of the next lecture, when you have the tools to answer them yourselves.

Practicalities

- Every lecture has a **Colab notebook**, linked from its slides
- Part I runs on free CPU. Parts II–VI need a GPU runtime, and each notebook says which
- Every deck has a PDF on the course page, one page per slide — or print this one from your browser
- Press `M` for the deck menu, `S` for speaker notes, `C` for the chalkboard

The rest of today

1. The brief — what we are being asked to build (10 min)
2. The data — look at it once, in full (15 min)
3. The split — and why it comes before the exploration (15 min)
4. Explore — properly, on the training set only (15 min)

GÉRON, CHAPTERS 1–2

The thirty minutes just spent were the course itself, and this is the only lecture that carries them. Every other lecture opens with ten minutes of context and twenty of mathematics.

PART ONE

The brief

8 minutes

You have just been hired

You are a data scientist at a real-estate investment company.

> **THE TASK**
>
> Use California census data to build a model of housing prices. The data gives population, median income and median housing price for each *block group* — the smallest unit the US Census publishes. We will call them **districts**.

Your model predicts a district's **median housing price** from the other metrics.

The first question is never technical

What exactly is the business objective?

- Building a model is not the goal
- How will the company use the output?
- The answer determines the framing, the algorithm, the metric, and how much effort is worth spending

What the output feeds

Your model's prediction is fed to **another machine learning system**, together with other signals, to decide whether a district is worth investing in.

> *TERMINOLOGY*
>
> *A piece of information fed to an ML system is called a signal — after Shannon's information theory. You want a high signal-to-noise ratio.*

✗ Note the consequence: nobody downstream ever looks at your prediction. A component that quietly degrades inside a chain of components is not noticed by anyone — which is why the evidence has to come from you.

What is the current solution?

District prices are currently estimated **manually by experts**: a team gathers information and applies complex rules.

> **THE NUMBER THAT MATTERS**
>
> When the team can check their estimates against the truth, they are typically off by more than **30%**.

Remember this figure — and read it carefully. It is a **relative** criterion: 30% *of the district's price*. In the next lecture we choose a metric measured in dollars, which is not the same thing, and we come back to this then.

It is also the stakeholder's claim about their own team, not a measurement we made. We will treat it as an assumption and label it as one wherever it appears.

Frame the problem

Before writing any code, answer four questions:

1. What kind of **training supervision** does this need?
2. Is it **classification, regression**, or something else?
3. Should we use **batch or online** learning?
4. What **assumptions** are we making?

Pause here and answer them yourself before the next slide.

Questions 1–3, answered

Question	Answer	Because
Supervision	✓ **supervised**	every district carries its expected output
Task	✓ **multiple, univariate regression**	predict one value, from several features
Learning mode	✓ **batch, offline**	no data stream, and 20,640 rows fit in memory

> ***THE ALTERNATIVES, FOR REFERENCE***
>
> *Unsupervised (no labels) · semi-supervised (some labels) · self-supervised (labels from the data itself) · reinforcement (rewards). Predict several values per district and it becomes multivariate regression; add a data stream and it becomes online learning.*

Three answers, one minute. The fourth question is the one that can sink the project.

4 · Check the assumptions

List and verify what everyone is assuming. This catches serious problems early.

> **THE ONE THAT WOULD SINK US**
>
> What if the downstream system converts our prices into *categories* — “cheap”, “medium”, “expensive” — and discards the numbers? Then this is a **classification** problem and we have built the wrong thing.

We asked. They need the actual prices. We may proceed.

PART TWO

The data

20 minutes · look, then split, then look properly

Automate the download

```
from pathlib import Path
import pandas as pd
import tarfile
import urllib.request

def load_housing_data():
    tarball_path = Path("datasets/housing.tgz")
    if not tarball_path.is_file():
        Path("datasets").mkdir(parents=True, exist_ok=True)
        url = "https://github.com/ageron/data/raw/main/housing.tgz"
        urllib.request.urlretrieve(url, tarball_path)
        with tarfile.open(tarball_path) as housing_tarball:
            housing_tarball.extractall(path="datasets", filter="data")
    return pd.read_csv(Path("datasets/housing/housing.csv"))

housing_full = load_housing_data()
```

Write a function, not a manual download: the data will change, and you will need this on another machine.

The structure

One row per district, ten attributes. Start with `head()` and `info()`, always, before anything else.

```
>>> housing_full.info()
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
 #   Column              Non-Null Count   Dtype
---  ------              --------------   -----
 0   longitude           20640 non-null   float64
 1   latitude            20640 non-null   float64
 2   housing_median_age  20640 non-null   float64
 3   total_rooms         20640 non-null   float64
 4   total_bedrooms      20433 non-null   float64
 5   population          20640 non-null   float64
 6   households          20640 non-null   float64
 7   median_income       20640 non-null   float64
 8   median_house_value  20640 non-null   float64
 9   ocean_proximity     20640 non-null   object
```

Observation: 207 districts are incomplete

`total_bedrooms` has **20,433** non-null values, not 20,640.

✗ 207 districts are missing this feature.

> **THREE OPTIONS, ALL LEGITIMATE**
>
> Drop those districts · drop the whole attribute · fill the gaps with a chosen value. We will return to this in the build.

One attribute is not numerical

```
>>> housing_full["ocean_proximity"].value_counts()
<1H OCEAN     9136
INLAND        6551
NEAR OCEAN    2658
NEAR BAY      2290
ISLAND           5
```

Repeated values from a small set: this is a **categorical** attribute, not free text.

✗ Note `ISLAND`: five districts. Rare categories will cause trouble later.

Summary statistics

```
housing_full.describe()
```

- `count` ignores nulls — that is why `total_bedrooms` reads 20,433
- `std` is the standard deviation: how dispersed the values are
- `25%`, `50%`, `75%` are the quartiles and the median

For a bell-shaped distribution the 68–95–99.7 rule applies: about 68% of values fall within 1σ of the mean, 95% within 2σ, 99.7% within 3σ.

Plot everything

```
import matplotlib.pyplot as plt

housing_full.hist(bins=50, figsize=(12, 8))
plt.show()
```

A histogram shows how many instances fall in each value range. Four things should jump out.

Nine attributes, fifty bins

Take thirty seconds. What is wrong with the bottom two?

Observation 1 · The income is not in dollars

`median_income` ranges roughly 0.5 to 15. That is not a salary.

> The data was scaled and capped: at 15 (actually 15.0001) at the top and 0.5 (0.4999) at the bottom. The numbers are roughly tens of thousands of dollars — 3 means about $30,000.

Preprocessed attributes are normal. **Understanding how they were computed is not optional.**

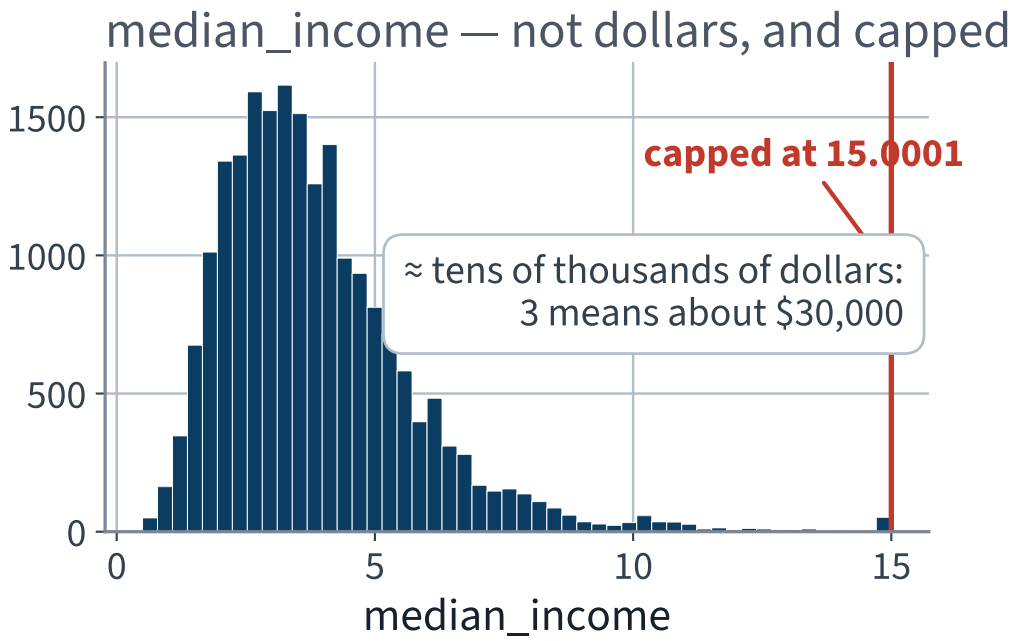
Observation 2 · The target is capped

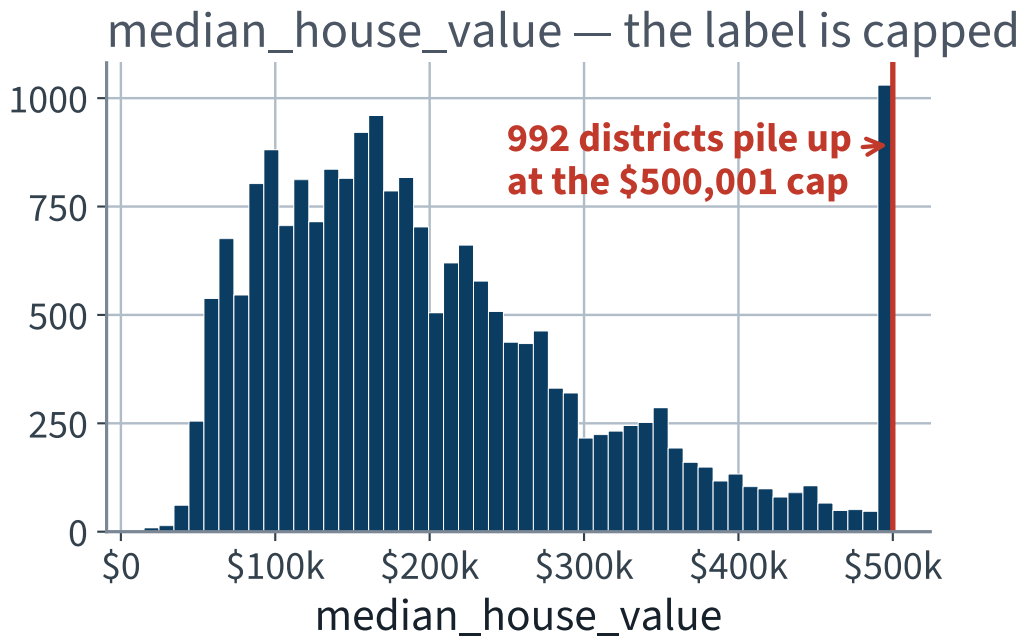
`median_house_value` is capped — and it is our **label**.

> **WHY THIS IS SERIOUS**
>
> Your model will learn that prices never exceed the cap. If the client needs accurate predictions beyond $500,000, the model is unusable there.

Two honest fixes: collect proper labels for the capped districts, or remove them from *both* the training and the test set.

Both spikes, enlarged





✗ 992 districts — nearly 5% of the data — have a label that is not their price. It is a ceiling.

Observation 3 · The scales differ wildly

`total_rooms` runs from 2 to 39,320. `median_income` runs from 0.4999 to 15.0001.

> **WHICH MATTERS FOR SOME METHODS AND NOT OTHERS**
>
> **Scaling is required for:** gradient descent, regularised linear models (ridge, lasso), SVMs, k-nearest neighbours, k-means, PCA, neural networks. Anything that measures a distance, penalises a coefficient, or takes a step.
>
> **Scaling does nothing for:** decision trees, random forests, and ordinary least squares. Trees split on one feature at a time, so any monotone rescaling gives the same tree; least squares is *equivariant* under an invertible affine map of the features — the fit is identical and the coefficients simply absorb the change.

✗ All three models you build today are in the second list. It is also exactly why the scale-before-split leak will cost nothing here: an invertible affine map, applied to an estimator that does not care. The next lecture measures it, over twenty seeds.

Observation 4 · Many attributes are skewed right

They extend much further to the right of the median than to the left.

This makes patterns harder for some algorithms to detect. Later we will transform them toward something more symmetric — typically by taking a square root or a logarithm.

A feature following a power law is a good candidate for $\log$: districts of 10,000 people are ten times rarer than districts of 1,000, not exponentially rarer.

Stop. Before we look any closer

Four observations from four histograms, and we have not yet plotted one variable against another. That is exactly the moment to stop.

The only way to know how a model generalises is to try it on cases it has never seen.

- Split the data: a **training set** and a **test set**
- The error rate on new cases is the **generalisation error**
- Low training error with high generalisation error means **overfitting**

Typically 80/20 — but with 10 million instances, holding out 1% is plenty.

Set it aside *now*, and do not look

DATA SNOOPING BIAS

Your brain is an outstanding pattern detector, which means it overfits. If you study the test set, you will pick a model that suits it, your estimate of generalisation error will be optimistic, and you will ship something that underperforms.

✗ Create the test set before you explore any further.

The obvious split, and its flaw

```
from sklearn.model_selection import train_test_split

train_set, test_set = train_test_split(housing_full, test_size=0.2,
                                       random_state=42)
```

Purely random sampling is fine *if the dataset is large enough relative to the number of attributes*.

✗ Ours may not be. Random sampling risks significant sampling bias.

Stratified sampling

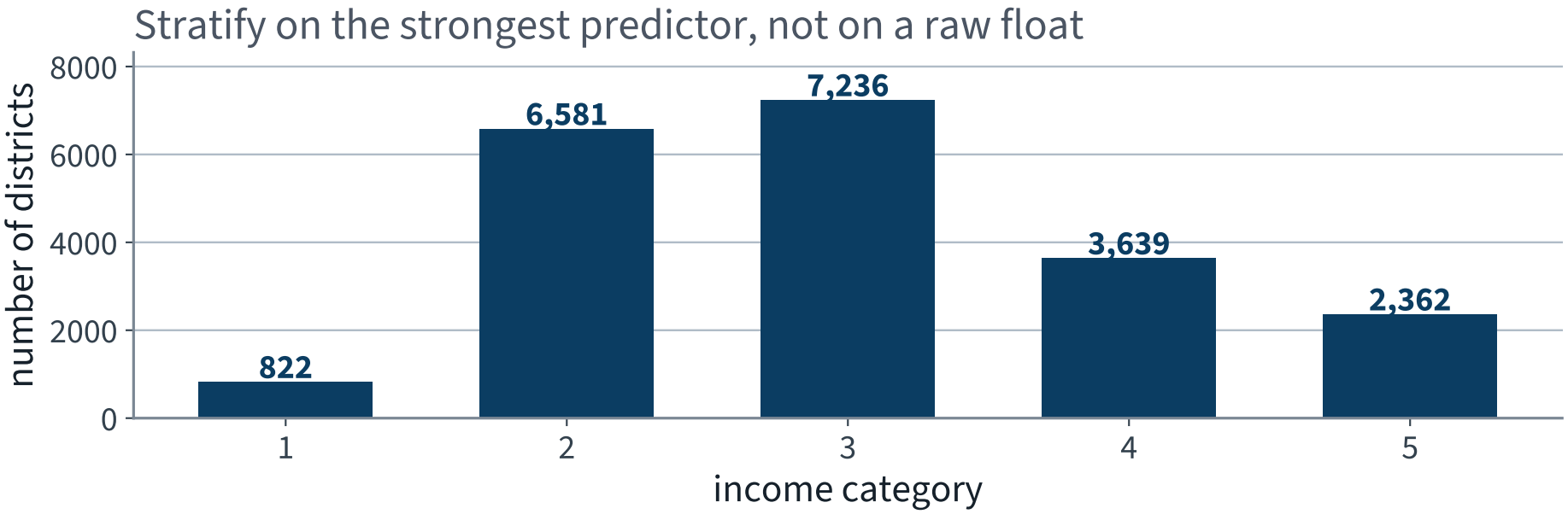
Surveyors do not call 1,000 people at random. They ensure the sample is representative of the population on the dimensions that matter.

Experts tell us **median income** predicts price. So our test set must be representative across income levels.

```
import numpy as np

housing_full["income_cat"] = pd.cut(housing_full["median_income"],
                                    bins=[0., 1.5, 3.0, 4.5, 6., np.inf],
                                    labels=[1, 2, 3, 4, 5])
```

Five strata, none of them tiny



The bins are chosen so that no stratum is too small to sample from. Category 1 has 822 districts — enough.

Split on the strata

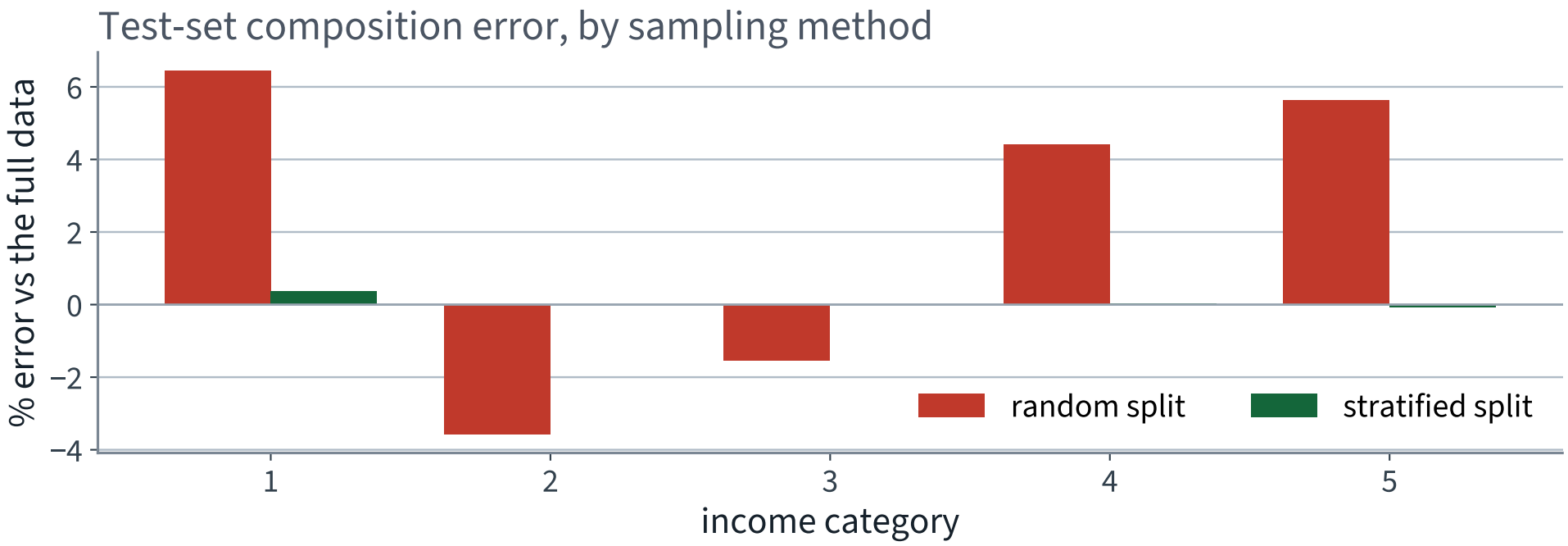
```
strat_train_set, strat_test_set = train_test_split(
    housing_full, test_size=0.2,
    stratify=housing_full["income_cat"], random_state=42)

for set_ in (strat_train_set, strat_test_set):
    set_.drop("income_cat", axis=1, inplace=True)
```

The stratified test set has income proportions almost identical to the full dataset. The purely random one is visibly skewed.

Not too many strata, and each stratum large enough — or the estimate of a stratum's importance is itself biased.

Does it actually matter? Measure it.



✗ Random split: up to 6.4% off. ✓ Stratified: up to 0.36% off. Roughly eighteen times better, for one keyword argument.

Check the split before you trust it

```
assert len(strat_train_set) + len(strat_test_set) == 20640
assert set(strat_train_set.columns) == set(strat_test_set.columns)
assert strat_train_set.index.intersection(strat_test_set.index).empty
```

Three lines: nothing was dropped, the two frames have the same columns, and no district is in both. Each one has been a real bug in a real notebook.

> **ASSERT AFTER EVERY STRUCTURAL STEP**
>
> An assertion is a claim about your data that fails *loudly*. It is the only thing in this course that does. Write one after every split, merge, reshape and drop — and count what you dropped.

Reviewer question 4 — *what was dropped?* — answered by the program instead of by you.

Now explore — the training set, and only that

```
housing = strat_train_set.copy()          # 16,512 districts. The other 4,128
                                          # are sealed until the last slide of
                                          # the next lecture.
```

Everything from here to the end of this part is computed on `housing`. Every correlation, every scatter, every ratio.

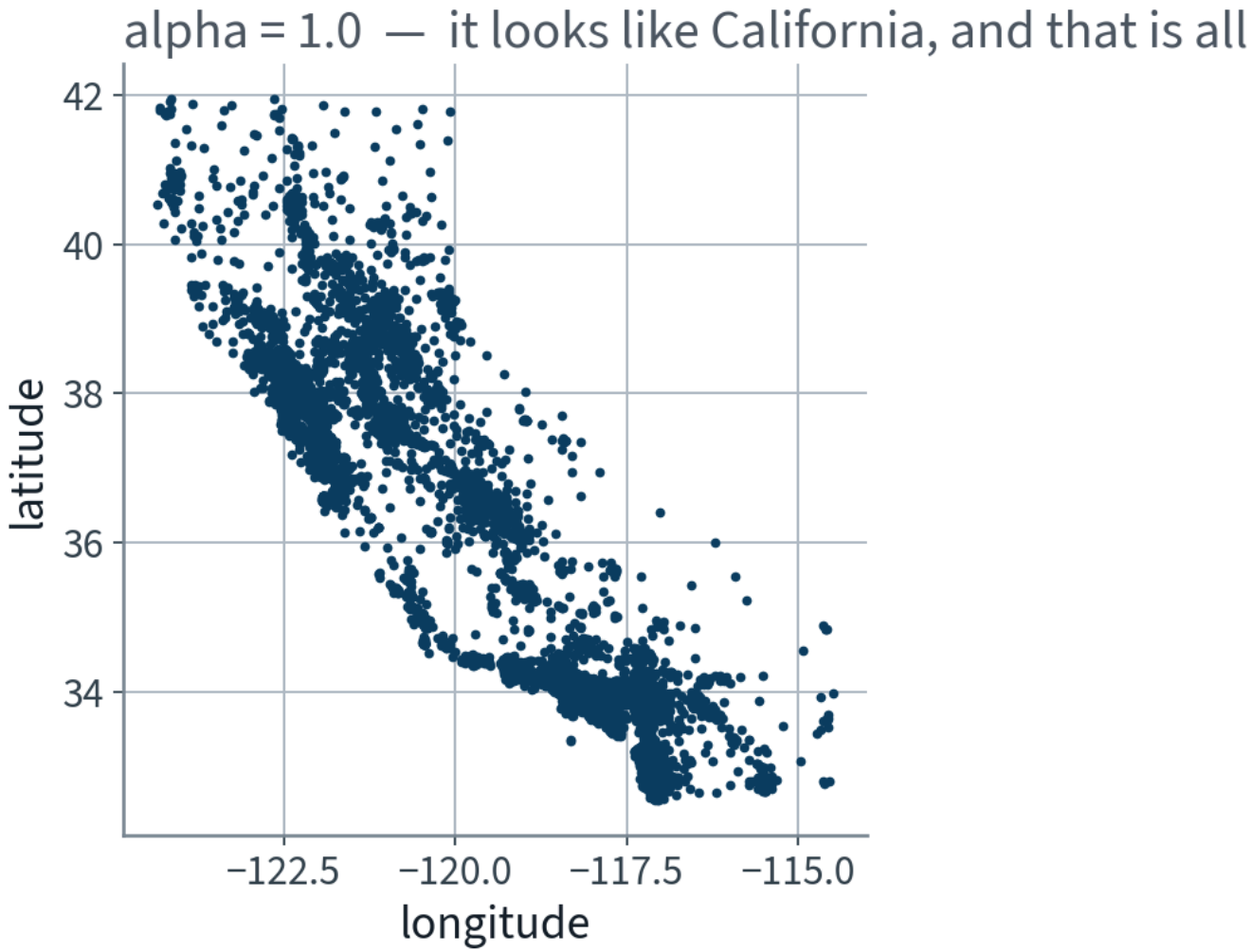
> **WHY THE COPY**
>
> So that nothing we invent while exploring — a new column, a filter, a dropped row — can reach `strat_train_set` by accident. And so that the numbers on the next eight slides are honestly labelled: they are **training-set** numbers, and they will differ slightly from anything you compute on all 20,640 rows.

The data is geographical — so look at it

```
housing.plot(kind="scatter", x="longitude", y="latitude", grid=True)
plt.show()
```

It looks like California. Beyond that, nothing is visible.

Every district, plotted



California. And nothing else — the ink is saturated.

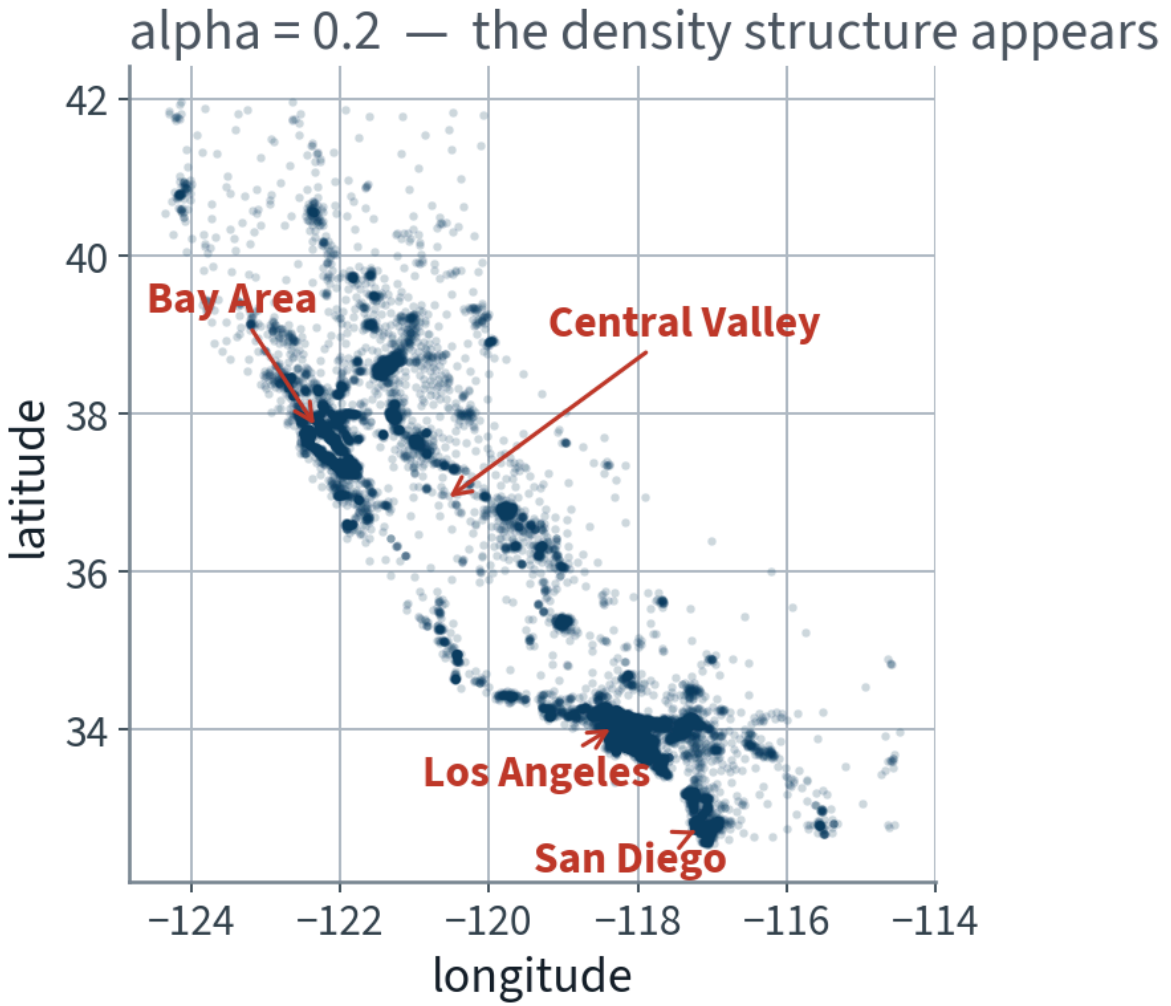
One parameter changes everything

```
housing.plot(kind="scatter", x="longitude", y="latitude",
             grid=True, alpha=0.2)
plt.show()
```

Now the high-density areas appear: the Bay Area, Los Angeles, San Diego, and the Central Valley around Sacramento and Fresno.

Our brains are excellent at spotting patterns in pictures. Expect to tune plotting parameters until the pattern shows.

The same data, at one fifth the opacity



One keyword argument. Same data, and now you can see the population of a state.

Three variables at once

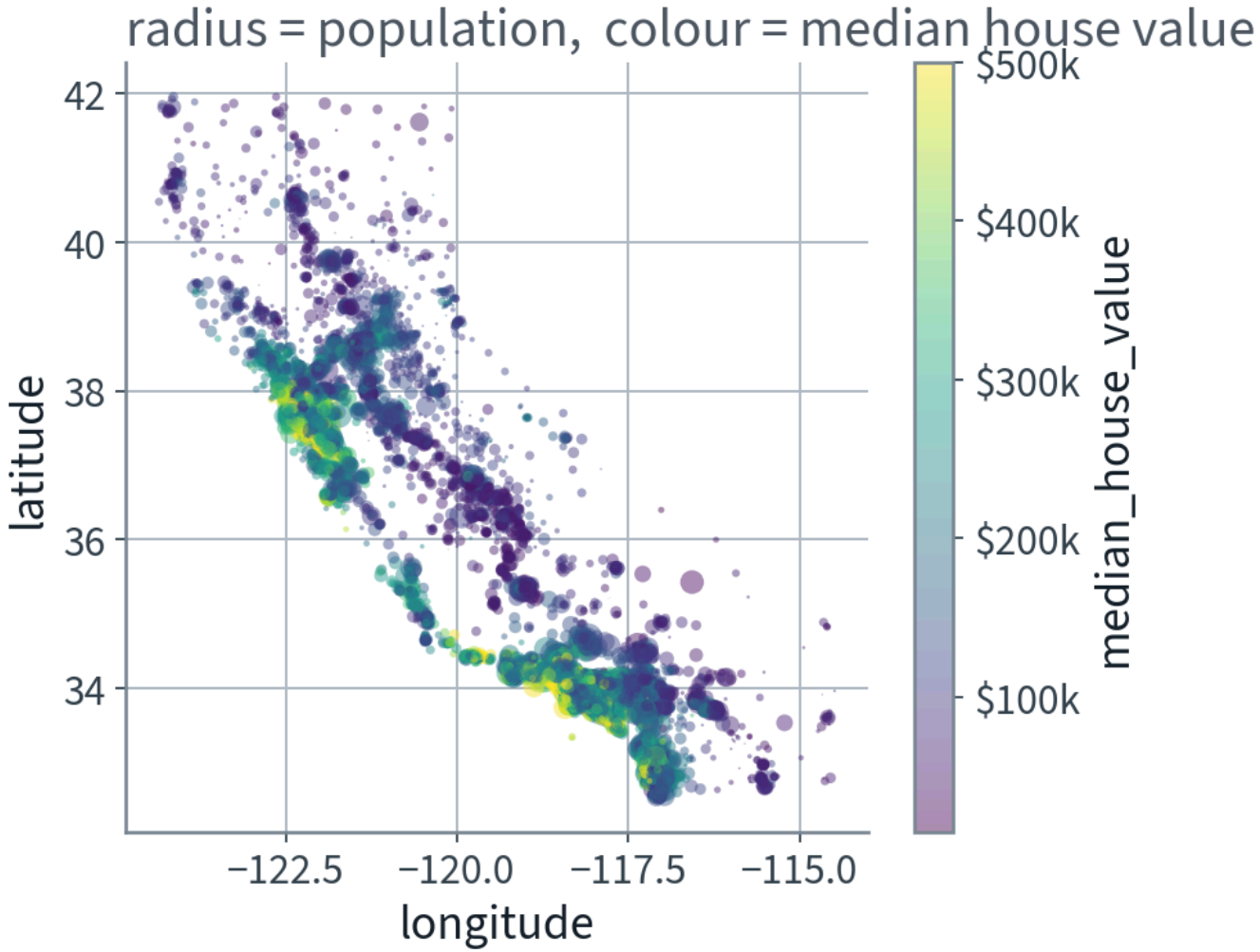
```
housing.plot(kind="scatter", x="longitude", y="latitude", grid=True,
             s=housing["population"] / 100, label="population",
             c="median_house_value", cmap="jet", colorbar=True,
             legend=True, sharex=False, figsize=(10, 7))
plt.show()
```

Radius encodes population, colour encodes price.

Prices are strongly related to **location** and **population density**. A clustering algorithm should be useful here — we will come back to that idea.

One line in that cell is worth arguing with, and we will — in the next slide but one.

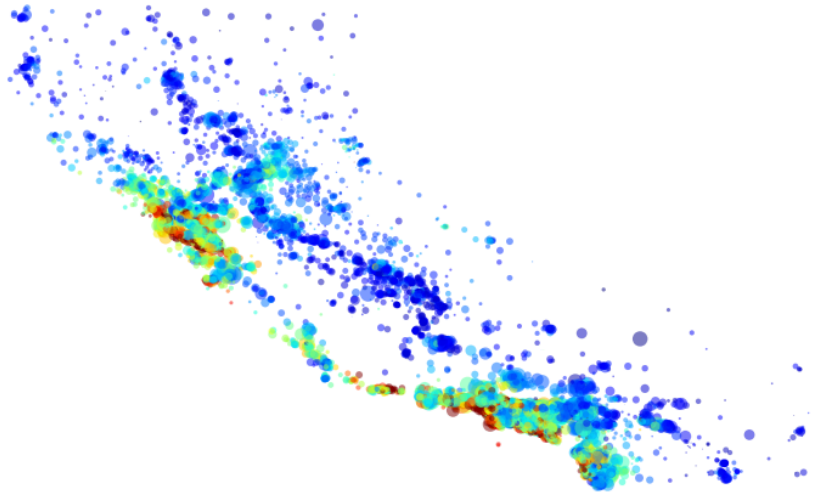
Location and density, against price

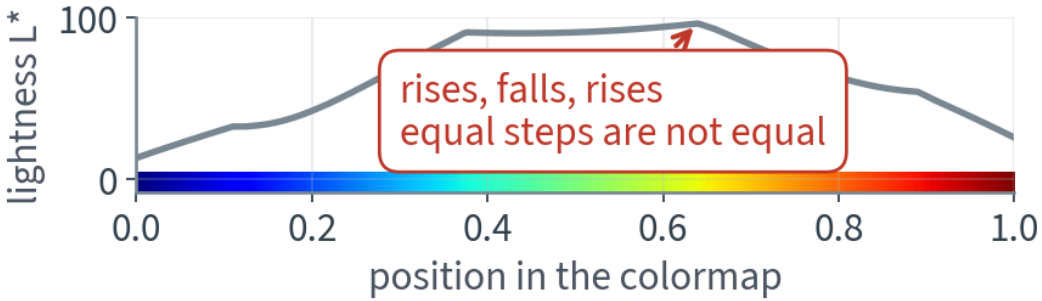


Red on the coast, blue inland. The model does not know what “coast” means — but it will learn longitude.

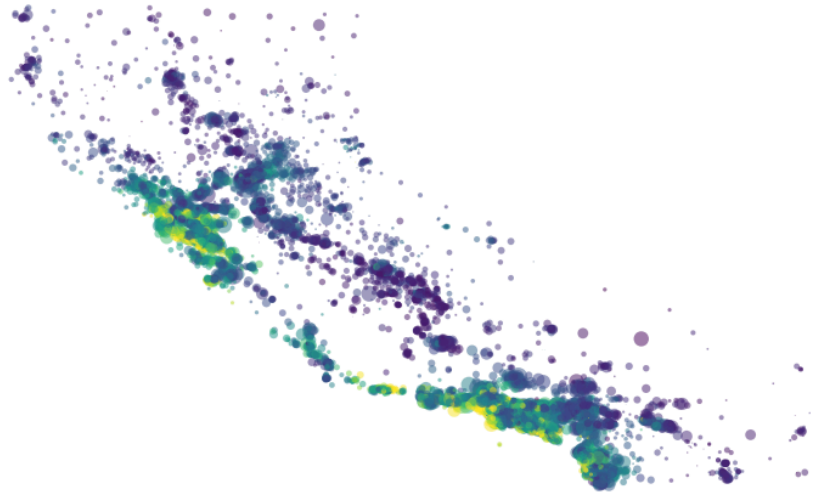
The one line worth arguing with

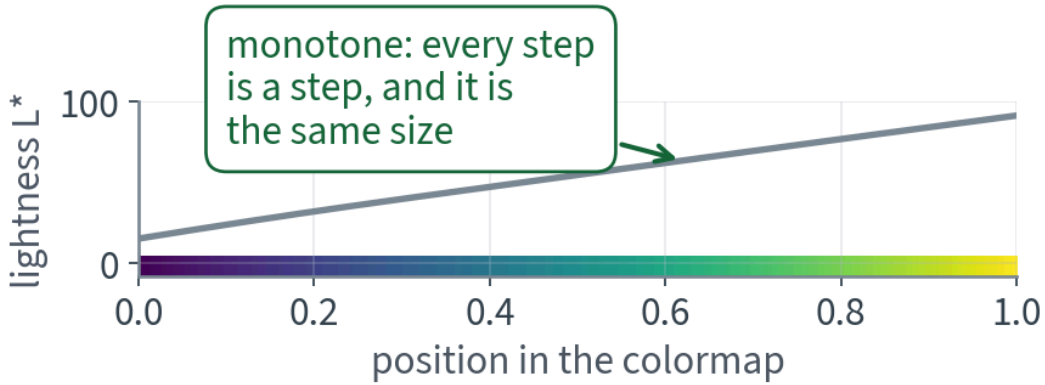
cmap="jet" — what the book writes





cmap="viridis" — what we plot





`cmap="jet"` ran, produced something colourful, and is wrong.

Why, and how you can check it yourself

A colour scale is a claim that equal steps in the data look like equal steps on the screen. The bottom row of that figure is that claim, tested: it plots each colormap's **lightness** against position.

Colormap	Lightness range	Largest reversal	Verdict
✗ `jet`	83.0	**✗ 1.06**	**✗ rises, falls, rises**
`turbo`	78.9	0.87	better, still not monotone
✓ `viridis`	75.9	**0.00 ✓**	**✓ every step is a step**

So `jet` invents bands the data does not have, buries detail in its yellow, and for the 8% of men with a red–green deficiency it reverses rank order outright. Photocopy it and it stops being a scale at all.

> **NOTICE THE SHAPE OF THIS**
>
> The code ran. Nothing errored. The defect was in a default nobody chose deliberately, and it took a measurement to see. **That is every failure in this course, and it is only the first lecture.**

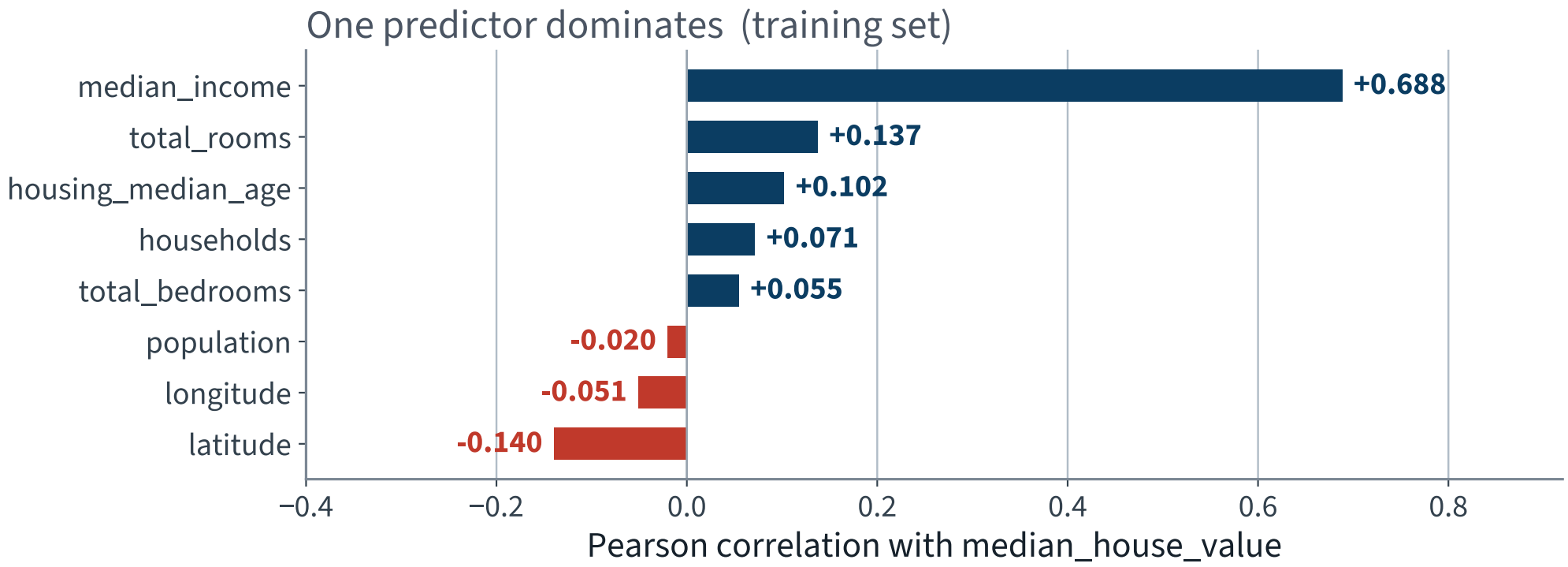
Standard correlation coefficient

```
>>> corr_matrix = housing.corr(numeric_only=True)
>>> corr_matrix["median_house_value"].sort_values(ascending=False)
median_house_value    1.000000
median_income         0.688380
total_rooms           0.137455
housing_median_age    0.102175
households            0.071426
total_bedrooms        0.054635
population           -0.020153
longitude            -0.050859
latitude             -0.139584
```

Pearson's r ranges from -1 to $+1$. Income dominates.

✗ These are training-set values. Compute them on all 20,640 rows and the third decimal moves — which is why a number is worth nothing without the set it was measured on.

The same column, as a picture



One bar is five times the next. That is the feature we already stratified on — on the experts' word, before we had looked. The measurement says they were right.

What the correlation coefficient cannot see

r measures **linear** correlation only.

> **TWO TRAPS**
>
> A dataset can have $r = 0$ and be completely dependent — any nonlinear relationship. And $r = \pm 1$ says nothing about the *slope*: your height in inches correlates 1.0 with your height in nanometres.

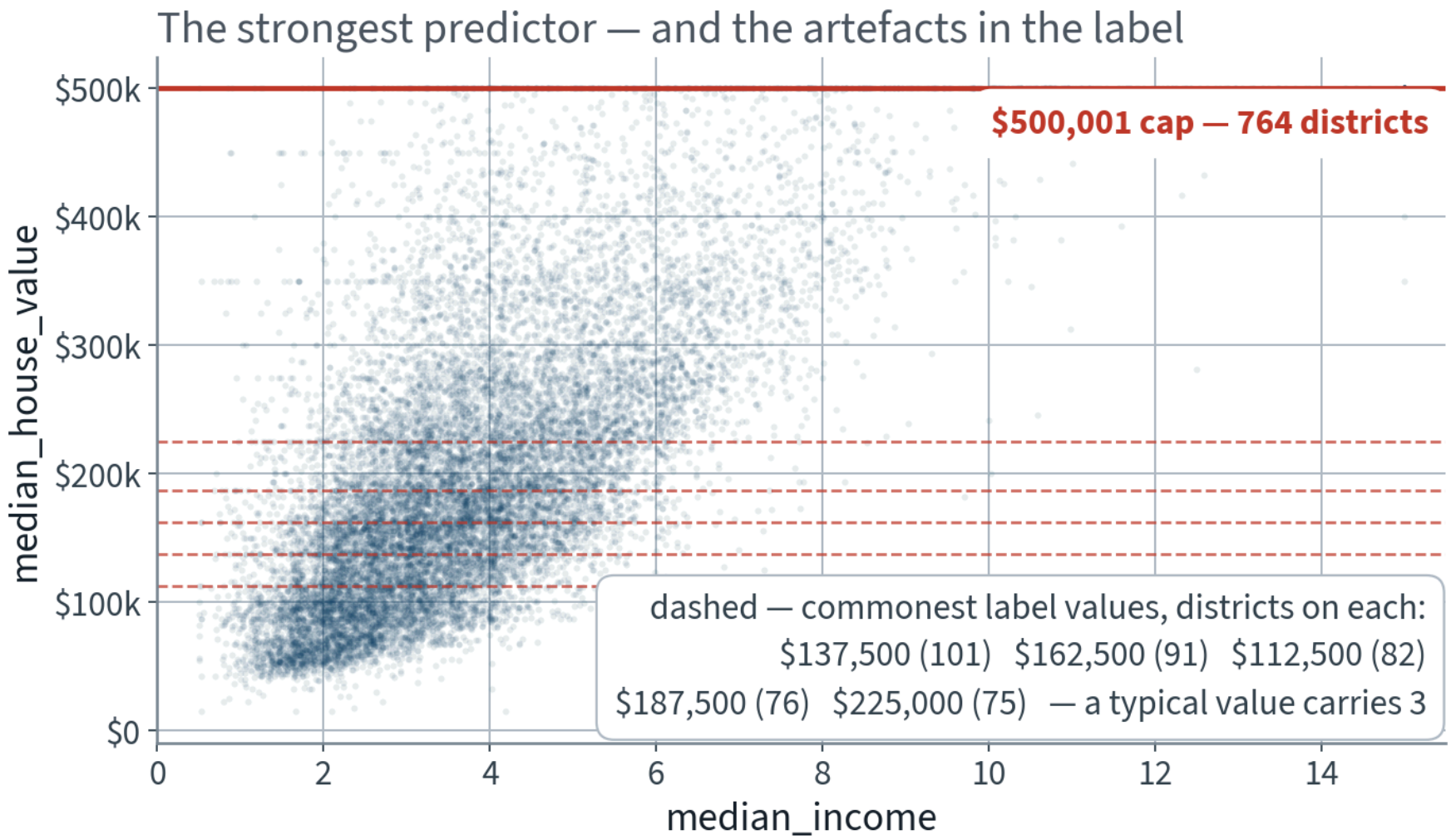
Zoom in on the strongest predictor

```
housing.plot(kind="scatter", x="median_income", y="median_house_value",
             alpha=0.1, grid=True)
plt.show()
```

- The upward trend is real, though noisy
- The $500,001 cap is a clear horizontal line — ***✗ 787 districts*** in our training split
- There are fainter lines too. **Which ones?** Do not squint at the plot — count.

A horizontal stripe means many districts share one exact price. That is a property of the table, not of California, so it is countable.

Income against price — and the lines that are really there



✗ **The trend is real. The stripes are not: every one sits on a multiple of $12,500.**

What the eye reported, and what the count says

A well-known description of this dataset names three fainter lines. We counted them in our own training split.

Value	Districts	Verdict
$350,000	62	**✓ real — 21× the background**
$450,000	31	marginal
$280,000	**✗ 3**	**✗ indistinguishable from background**

A typical price is shared by **3** districts, so the last one is not a line at all. Meanwhile the five commonest values below the cap — $137,500, $162,500, $112,500, $187,500, $225,000 — go unmentioned, and they are the strongest stripes in the plot.

THE HABIT, NOT THE NUMBERS

Three values were asserted; one survives. Counting took one line. You will inherit descriptions of data for the rest of your career — check them against the data before you build on them.

Combine attributes before modelling

```
housing["rooms_per_house"]  = housing["total_rooms"] / housing["households"]
housing["bedrooms_ratio"]   = housing["total_bedrooms"] / housing["total_rooms"]
housing["people_per_house"] = housing["population"] / housing["households"]
```

Attribute	Correlation with price
median_income	0.688
rooms_per_house	0.144
total_rooms	0.137
bedrooms_ratio	**−0.256 ✓**

`bedrooms_ratio` is far more informative than either raw count.

A warning you will need in Lecture 2

COLLINEARITY

When you create combined features, make sure they are not too linearly correlated with existing ones. In particular, **avoid simple weighted sums of existing features.**

Collinearity causes real problems for some models — linear regression above all.

In the next lecture we will see *exactly* why, when we derive the normal equation.

The notebook for this lecture

Open Lecture 1 in Colab

- **Runs on free CPU**, about two minutes end to end
- Everything on today's slides, in order: fetch, inspect, the missing values, the histograms, the stratified split, and the exploration
- Every code cell carries the specification that would produce it — read the box, predict the output, then run

> **WHAT TO DO WITH IT BEFORE THE NEXT LECTURE**
>
> Run it top to bottom once. Then change one thing — the number of income bands in `pd.cut` — and see what happens to the sampling bias table. Two lines, and it is the whole argument of the stratification section.

What we did

1. Set out what the course is, how a lecture is shaped, and what the examination asks for
2. Set out the working method: **specify, generate, read, test, verify** — and that steps 3–5 are the course
3. Framed the problem before touching the data — supervised, multiple univariate regression, batch
4. Checked the assumptions with the downstream team, and found the output is consumed as a *price*, not as a category
5. Looked at the whole dataset once: missing values, a capped target, mismatched scales, skew
6. Built the stratified split, and *then* explored — on the training set only
7. Found the strongest predictor, the survey artefacts in the target, and three engineered ratios worth more than the columns they came from

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 1

five, in the style of the written paper

Exercises — Lecture 1

1 A colleague reports that their model reaches an RMSE of $48{,}000$ on the California housing data, and that they chose the model by trying six of them and keeping the best. State, in one sentence, what that number is an estimate of — and what it is *not* an estimate of. [4]

2 The median income column is capped at 15. Give one consequence for the *model* and one for the *evaluation*, and say which of the two you would raise with the client first. [5]

3 You are given a stratified split on income category and a naive random split of the same data. Both report similar RMSE. Does that mean the stratification was unnecessary? Answer yes or no, with a reason. [4]

Solutions on Lecture 2's deck.

Exercises — Lecture 1 (continued)

4 The brief says the output feeds a downstream system that expects a price. Name the one thing this fact settles about the problem framing, and one thing it leaves open. [4]

5 In the working loop — specify, generate, read, test, verify — exactly one step is done by the assistant. Name it, and say what goes wrong if a student treats a second step as the assistant's. [3]

Solutions on Lecture 2's deck.

LECTURE 2 · GÉRON CH. 2

The end-to-end project

Least squares and the normal equation — and why you avoid weighted sums of existing features
Pipelines, `ColumnTransformer`, cross-validation, grid search
Choosing RMSE or MAE, and the final evaluation on the sealed test set

Run the Lecture 1 notebook first